CAMPUSWEAVE / ACADEMIC PLANNING

Make room for learning.

College timetable planning system

Project report, implementation evidence and complete DBMS syllabus workbook.

RELATIONAL	NON-RELATIONAL	PL/SQL
MySQL 8.4	MongoDB 8.0	Oracle 21c verified

Prepared for academic demonstration • 1 November 2026

Source: github.com/malpurearchit25/CampusWeave

Reading guide

Part	Contents
1 / Evaluation	Rubric evidence map and observed verification results
2 / Project report	Requirements, ER mapping, normalization, SQL, transactions, architecture, demonstration and viva
3 / Syllabus workbook	All six units and ten laboratory experiments, with implementation boundaries
4 / Visual evidence	Interface screenshots, operational ER overview, detail diagrams and warehouse
5 / Data dictionary	Every physical table and its declared columns, nullability and keys

Rubric evidence map

Rubric	Marks	Concrete evidence
Requirement analysis and complete ER diagram	15	PROJECT_REPORT sections 1-2; ER_complete.svg; academic/scheduling detail diagrams; metadata-derived dictionary.
Schema design and normalization	15	sql/01_schema.sql; report section 3; candidate keys, functional dependencies, 1NF-3NF decomposition and mapping table.
SQL and PL/SQL implementation	20	Query showcase; sp_validate_run; sp_publish_run called by the UI; insert/update/delete guards; automatic audit trigger; separate Oracle PL/SQL lab.
Query optimization and transaction processing	15	Controlled 60,000-row EXPLAIN benchmark; evidence.json; live rollback/savepoint lab; failed-write rollback and concurrent publication tests.
Architecture and application functionality	15	architecture.svg; running Flask/MySQL application; authenticated catalog CRUD; cross-semester scheduler; weekly grid, publication, archive, CSV and print; tests.
Report, demonstration and viva	20	CampusWeave_Report.pdf; editable PROJECT_REPORT.md; eight-minute demo plan; viva answers; UI screenshots and test results.

This is an evidence map, not a promise of marks. The institution decides scoring. MySQL and Oracle dialects are clearly distinguished; the Oracle companion has been executed on Oracle 21c and includes rerunnable evidence.

Verification record

Verified on 1 November 2026. Application: Flask/Waitress, MySQL 8.4.13, MongoDB 8.0.29.

Check	Observed result
Automated integration suite	51 passed, 0 failed, 0 skipped; 38.632 seconds in the recorded full run.
MySQL syllabus queries	50 executed in an isolated schema; role grant/revoke and checked-view rejection also verified.
MongoDB laboratory	16 operations executed, including CRUD, aggregation, indexing and legacy MapReduce.
Oracle 21c companion	42 batches; no unhandled errors; expected capacity rejection; six cursor trace rows; empty compiler error results.
Index experiment	60,000 isolated rows; EXPLAIN changed from ALL/60,000 estimated rows to ref/1, with identical output.
Normalization and distributed models	FD closure, synthesis, serializability, recovery, fragmentation reconstruction, 4NF decomposition and parallel/serial totals checked.
Browser workflow	Sign-in, generation, review and publication; visible publication success, complete practical blocks, 19-trigger inventory, rollback result 1/1/0, warehouse total 32 hours, and a narrow home layout with no document overflow.

Full test names are in test_evidence.json. Machine-readable evidence is stored beside this document. The Oracle transcript and public result are linked in the README. Durations are observations, not performance guarantees.

Presentation checklist

1. Start the application and sign in.
2. Show the ER diagram, session/period decomposition and eligibility junction.
3. Review academic assignments and generate a draft for an unpublished scope.
4. Show contiguous practicals, then publish and inspect the audit record.
5. Show published-only insights and load the warehouse once.
6. Run the rollback experiment and explain the 1, 1, 0 result.
7. Show MongoDB feedback and syllabus experiments.
8. Open the Oracle evidence and explain one cursor, function and trigger.

Prepared PC note: semester 3 run 2 is published for a ready-to-show example. Semester 3 cannot be generated again until that edition is archived; semester 5 remains available for a fresh generation demonstration.

CampusWeave

A faculty-friendly college timetable planning system

DBMS course project | Flask + MySQL 8.4 | Academic edition 2026-27

CampusWeave turns academic assignments into a validated weekly timetable. It combines a normalized relational schema, a bounded constraint solver, stored procedures, database triggers, and atomic publication. A faculty-oriented interface keeps routine scheduling separate from the DBMS demonstration lab.

This report describes the implemented system and its observed tests. It does not claim that a feasible schedule is globally optimal or that the project is ready for unrestricted internet deployment. Sample names and academic data are illustrative.

1. Requirement analysis

Problem statement

Manual timetabling distributes the same academic facts across spreadsheets and informal messages. A faculty member may be assigned to two divisions simultaneously; a practical may be placed in an unsuitable room; changing one semester may invalidate another. The problem is to assign every required course session to a day, consecutive teaching periods, and a suitable room while respecting shared resources.

Actors and boundaries

The demonstrated actor is an authenticated academic coordinator. The seeded admin account performs catalog management, generation, publication and archival. Faculty are scheduling resources and report consumers; separate faculty and student portals are outside the implemented scope. The users table retains role metadata, but this is not a complete multi-role authorization product.

The application runs on this PC at 127.0.0.1:5050. Its MySQL server listens only on 127.0.0.1:3307. One academic year is treated as one scheduling conflict domain: published semesters in the same year share room and faculty reservations. Independent date ranges, rotating weeks, electives and team teaching are not modeled.

Functional requirements and acceptance criteria

ID	Requirement	Observable acceptance criterion
FR1	Maintain academic entities	Create, inspect, edit and remove unreferenced catalog records through validated forms.
FR2	Enforce teaching eligibility	An assignment requires an existing faculty-course pair through a composite foreign key.
FR3	Generate all required sessions	A draft contains sessions_per_week times duration periods for every scoped offering.
FR4	Avoid resource clashes	Faculty, division and room cannot overlap within a run or against another published run in the same year.
FR5	Respect constraints	Availability, exact required laboratory type, capacity and combined workload are checked.
FR6	Keep practicals together	Each session has a session_no and occupies consecutive one-hour teaching periods in one room and day.
FR7	Spread repeated sessions	At most one session of an offering is placed on a given day.
FR8	Publish safely	Database validation, status update and audit insertion succeed or roll back together.
FR9	Preserve history	Published runs can be archived; their entries and source records remain protected.
FR10	Explain and export	Weekly grid, selected-division print/PDF, whole-run CSV, workload and room occupancy reports are available.
FR11	Demonstrate DBMS concepts	Live savepoint/rollback lab, procedure and trigger inventory, and measured EXPLAIN evidence are supplied.

Non-functional requirements

Consistency is enforced in MySQL as well as in the scheduler. All application writes use a shared scheduling lock to serialize configuration changes and publication. Form submissions use CSRF tokens; templates escape values; SQL values use bound parameters. Credentials are kept outside Git. The browser UI uses local fonts and assets, so the demonstration requires no CDN or external API.

The search has an eight-second/100,000-node budget and a 200-session limit. These are protective bounds, not proof of infeasibility when a limit is reached. The seeded semester-3 case contains 28 sessions and 32 occupied periods; tests verify complete generation.

2. Conceptual model and ER-to-relational mapping

The complete ER diagram is supplied as ER_complete.svg and ER_complete.png. Academic and scheduling detail diagrams provide larger views. They are generated from the installed MySQL metadata rather than an unrelated conceptual sketch. There are 14 business tables and one technical scheduling_lock table, plus five separate dimensional warehouse tables. timetable_entries is a flattened read-only projection and faculty_schedule_view is a reporting view; neither is a stored entity.

Mapping rules

Strong entities become tables with surrogate primary keys and natural alternate keys where appropriate. A one-to-many relationship places its foreign key on the many side. A many-to-many faculty-course relationship becomes faculty_courses with composite primary key (faculty_id, course_id). The optional faculty-user link is represented by a nullable unique foreign key. No repeating comma-separated list is stored in a column.

Parent / relationship	Child	Cardinality and implementation
departments	faculty, courses, divisions, timetable_runs	One department to zero or many rows; each child requires one department.
users	faculty	Optional one-to-one; faculty.user_id is nullable and unique.
faculty and courses	faculty_courses	Many-to-many resolved through the pair key.
faculty_courses	course_offerings	One eligible pair to many offerings; composite foreign key enforces eligibility.
divisions and courses	course_offerings	Each offering references one division and one course; their pair is unique.
faculty and time_slots	faculty_unavailability	A faculty/day/slot triple is unique.
users	timetable_runs	One creator per run; creator can own many runs.
timetable_runs	timetable_sessions	One run contains many numbered teaching sessions.
course_offerings and rooms	timetable_sessions	Each session references one offering and room; day is stored once.
timetable_sessions and time_slots	timetable_periods	Each occupied-period row references one session and one teaching period.
timetable_runs and users	approval_history	Each status action records its run, actor and timestamp.

The dictionary appendix lists every column, SQL type, nullability, primary/foreign key and alternate unique key. The diagram's question mark denotes a nullable field. A session is identified by (run_id, offering_id, session_no); its day and room are stored once in timetable_sessions. timetable_periods stores only entry_id, session_id and slot_id. The flattened timetable_entries view makes existing reports convenient without repeating stored facts.

3. Normalization through 3NF

Begin with an unnormalized scheduling sheet

Imagine a row containing DivisionName, DepartmentName, AcademicYear, Semester, StudentCount, CourseCode, CourseName, FacultyCode, FacultyName, RoomCode, RoomCapacity and a list of weekly periods. Repeating period lists violate 1NF. Course titles, faculty names and room capacity are copied across many rows, creating update anomalies. Deleting the only scheduled occurrence of a room could also delete the only record of its capacity.

First normal form

Expand the repeating period list into individual occupied-period rows. Each field contains one value. A candidate occurrence key is (RunID, OfferingID, Day, SlotID). This alone does not remove repeated course, faculty and room facts.

Second normal form

In that flattened relation, OfferingID determines DivisionID, CourseID, FacultyID and weekly requirements. RunID determines the run's academic scope and status. These depend on only part of the composite occurrence key. Move them into course_offerings and timetable_runs. Separate a session header (run, offering, session number, room and day) from its individual slot allocations.

The faculty_courses relation has only its composite key and no non-key attributes, so no partial dependency exists. Single-column primary keys in the entity tables also have no partial primary-key dependency; this observation alone does not prove 3NF.

Third normal form

Remove transitive descriptive dependencies: CourseID determines CourseName and SubjectType; FacultyID determines FacultyName and DepartmentID; DepartmentID determines DepartmentName; RoomID determines Capacity and RoomType; SlotID determines StartTime and EndTime. Store those attributes in their respective entity tables and reference them using foreign keys.

Relation	Principal functional dependencies / candidate keys
departments	department_id -> department_name; department_name is an alternate key.
users	user_id -> username, password_hash, role, is_active; username is an alternate key.
faculty	faculty_id -> code, name, department, designation, max hours, optional user; faculty_code is an alternate key. The nullable user link is a conditional unique identifier, not an unconditional candidate key.
divisions	division_id -> all attributes; (division_name, department_id, academic_year, semester) is an alternate key.
courses	course_id -> code, name, department, type, default sessions, duration, required room type; course_code is an alternate key.
rooms	room_id -> code, name, type, capacity, active; room_code is an alternate key.
time_slots	slot_id -> start_time, end_time, slot_type; start_time and end_time are alternate keys under the enforced fixed-hour, unique-period rules.

Relation	Principal functional dependencies / candidate keys
faculty_courses	(faculty_id, course_id) is the key; no non-key attributes.
course_offerings	offering_id -> division, course, faculty, weekly sessions; (division_id, course_id) is an alternate key.
faculty_unavailability	unavailability_id -> faculty, day, slot; (faculty_id, day_of_week, slot_id) is an alternate key.
timetable_runs	run_id -> scope, status, creator, creation time, search_nodes.
timetable_sessions	session_id -> run, offering, session_no, room, day; (run_id, offering_id, session_no) is an alternate key.
timetable_periods	entry_id -> session_id, slot_id; (session_id, slot_id) is an alternate key.
approval_history	approval_id -> run, actor, action, comment, action time.
scheduling_lock	lock_id only; technical singleton used for serialization.

The session/period split removes the dependency (run, offering, session_no) -> room, day from the occupied-period table. Keeping those repeated columns in each physical period row would violate 3NF. Repetition in a reporting view is harmless because it is derived. Under the stated business dependencies, each non-key attribute depends on a candidate key, the whole key, and no non-key determinant. The decomposed business relations satisfy 3NF. The design does not assert that all possible future college policies preserve these dependencies.

Why two sessions_per_week columns are intentional

courses.sessions_per_week is the subject's default requirement. course_offerings.sessions_per_week is the actual requirement for one division. An offering may override the default; therefore CourseID does not determine the offering-level weekly requirement. These are different facts, not accidental duplicate storage. Scheduling and completeness checks use the offering value.

Lossless decomposition and anomaly removal

Each entity decomposition retains the determining key as the common attribute: for example, the course projection and offering relation share CourseID, which is a key of courses. Joining them restores the descriptive data without inventing course identities. Foreign keys preserve referential integrity; the eligible pair dependency is preserved through a composite reference. A course-name correction is made once before the course is used in a saved edition. Saved editions protect referenced source records so their historical meaning cannot silently change.

4. SQL, stored procedures and triggers

The application uses MySQL stored SQL/PSM. Oracle PL/SQL is a different dialect. A separate, clearly labeled Oracle companion lab is included at sql/oracle/plsql_lab.sql. It demonstrates a capacity trigger, publication procedure, audit trigger, exception handling and transaction rollback. It is not a port of the complete application and has been executed on Oracle 21c in the approved CC0 db<>fiddle environment. The 42-batch evidence and public rerunnable link are included in oracle_execution.md.

Relational SQL coverage

sql/04_query_showcase.sql contains multi-table joins, LEFT JOIN with zero-count handling, aggregation, HAVING, correlated NOT EXISTS, a reporting view, a CTE with DENSE_RANK, EXPLAIN and metadata inspection. The UI exercises INSERT, SELECT, UPDATE and DELETE for eligible catalog records. SQL values are parameterized; dynamic identifiers come only from the catalog allowlist.

sp_validate_run

The validation procedure rejects an absent or empty run, missing assignments, incomplete weekly requirements, invalid academic scope, unsuitable or inactive rooms, unavailable faculty, malformed consecutive blocks, repeated same-day sessions, resource collisions and combined published workload overflow. It uses SIGNAL SQLSTATE '45000' to return domain errors to Flask. A valid call produces no result set.

sp_publish_run

The publication procedure takes a run ID and actor ID. It locks the singleton scheduling row and target run, checks that the actor is an active coordinator-level account and that the run is a draft, rejects a second active timetable for the same cohort, calls validation, then updates status. It does not COMMIT internally: the caller owns the transaction, including the audit trigger's insert.

Example: START TRANSACTION; CALL sp_publish_run(1,1); COMMIT; Use an actual draft ID from the dashboard. A failed call must be followed by ROLLBACK in an interactive SQL client. The Python transaction context performs that rollback automatically.

Trigger coverage

trg_session_insert guards session headers against invalid scope/session numbers, insufficient capacity and wrong room type. trg_validate_timetable_entry guards period allocations against unavailable faculty, break periods and same-run resource collisions. trg_entry_immutable prevents update-based bypass. trg_entry_delete protects published and archived entries. trg_run_transition checks permitted lifecycle transitions and revalidates publication. trg_run_audit records status changes automatically.

Catalog-history triggers reject changes to faculty, subjects, divisions, rooms or assignments used by saved schedules. Availability additions cannot contradict a published schedule; availability edits use remove-and-recreate. Period triggers reject overlapping times. Foreign keys reject deletion of referenced parent rows. Application management writes share the scheduling lock; unrestricted database-admin edits remain outside the supported UI workflow.

5. Query optimization and transaction processing

Query optimization method

The normalized application indexes session headers by (run_id, day_of_week), (room_id, day_of_week), and offering; occupied periods have a unique (session_id, slot_id) index and a slot foreign-key index. The separate index microbenchmark uses a flattened synthetic table to isolate the effect of (run_id, day_of_week, slot_id). Scope indexes support department/year/semester and published-year lookups. Composite index ordering follows equality predicates used for timetable retrieval. Indexes consume storage and add write cost; they are not added to every column indiscriminately.

The benchmark script creates a uniquely named disposable database with 60,000 synthetic entry rows, executes the identical selective query seven times without the composite index, adds the index, analyzes the table, then repeats the measurement. Both variants return exactly one row. The isolated database is removed afterward. The report's evidence appendix contains the actual EXPLAIN plans, estimated rows and median wall-clock timings from this PC.

The unindexed plan uses ALL and estimates 60,000 rows. The indexed plan uses ref and estimates one row. Timings are environment- and cache-dependent; the test demonstrates index access selection rather than a universal speed guarantee. The small seeded production dataset may reasonably choose different plans.

Atomicity

Generation inserts one run and all its entries within a single transaction. If a trigger rejects an entry, the exception rolls back the entire operation. Publication updates the run and inserts history through a trigger in the same transaction. A test deliberately inserts a valid entry followed by a conflicting entry and verifies that neither the run nor its earlier entry remains.

Consistency

Primary keys, foreign keys, unique constraints, CHECK constraints, validation procedures and triggers enforce domain invariants. The scheduler plans constraints in memory; the database independently rechecks the persisted result. Completeness is checked at publication because an intermediate insertion state is necessarily incomplete.

Isolation and concurrency

InnoDB provides transactional row locks. All application mutations acquire SELECT ... FOR UPDATE on scheduling_lock(lock_id=1). Generation holds this lock through planning and persistence; the search budget limits how long it can block other writers. Publication revalidates while holding the same lock. A two-connection concurrent publication test starts with two valid drafts for the same cohort and verifies exactly one publication and one audit record.

The coarse lock is a deliberate demonstration-scale tradeoff: easy-to-explain correctness over maximum parallel throughput. A larger institution would partition locks by conflict domain and implement deadlock retry. Readers may view drafts during planning but cannot use a stale browser validation result to bypass publication-time checks.

Durability and savepoints

Successful transactions call COMMIT and persist in InnoDB. Durability also depends on server configuration and storage; this project does not claim a power-loss test. The DBMS Lab inserts a temporary department, creates a savepoint, inserts a second department, rolls back the second insert, then rolls back the entire transaction. Counts observed at each stage are 1, 1 and 0. No experiment records remain.

6. Architecture and application functionality

The architecture diagram separates the browser, Flask/Waitress request layer, bounded scheduler and MySQL persistence. app.py handles HTTP requests and renders Jinja templates. catalog.py defines the only allowed entities and fields. db.py manages parameter binding, connection lifetime and commit/rollback. scheduler.py reads constraints, constructs a feasible plan and persists it atomically.

Scheduling algorithm

Expand each offering into numbered sessions. Sort long blocks first. Enumerate days ordered by the current division's daily load, then contiguous teaching blocks and suitable rooms. Reject blocked faculty periods, over-limit workload and overlapping faculty/division/room reservations. Repeated sessions of the same offering use different days. Backtrack when a placement blocks a later task. The first complete feasible assignment is returned; search budget exhaustion is reported separately from a proven exhausted search.

One-hour periods make occupied-period counts equal teaching hours. A practical requires its configured room type rather than any arbitrary lab. Published runs in the same academic year reserve resources across all departments and semesters. A new draft is not itself a reservation; two drafts may coexist, but publication rechecks their conflicts.

User workflow

Sign in using the local demo account. Review assignments and faculty eligibility. Select a cohort and generate a draft. Inspect the weekly grid by division; multi-period sessions are shown as a single spanning block. Publish after review. Export CSV for all divisions or print the selected division to PDF. Insights reports only published hours in the selected academic year. Archive a published edition to release reservations while retaining its history.

Interface and security decisions

The faculty interface uses navy, blue, ivory and restrained gold. Subject colors support scanning without being the only source of meaning; each cell also names its subject and room. Form labels, keyboard focus, a skip link, readable errors, mobile navigation and a print layout support everyday use. Server-side validation remains authoritative.

Passwords are stored as hashes. The demo password is intentionally documented for local evaluation, not a production secret. Sessions use HttpOnly and SameSite=Lax cookies; POST forms require CSRF tokens. Templates escape untrusted values, CSV text is guarded against formula injection, and a Content Security Policy limits resources to the local application. The server binds to loopback. Public hosting would require separate deployment, stronger account management, HTTPS and additional review.

7. Verification and results

The supplied pytest suite creates a fresh, uniquely named database per test. It never resets the user's application database. Tests cover valid generation, empty/incomplete schedules, wrong lab type, unavailable faculty, duplicate resource allocation, update bypass, failed multi-write rollback, protected source records, cross-semester scheduling, concurrent publication, archival, immutable published entries, infeasible workload, overlapping periods and eligibility foreign keys.

Web tests cover all main pages, CSRF rejection, login, output escaping, catalog create/edit/delete, savepoint cleanup, publishing through the stored procedure, CSV row counts, missing-run handling and draft exclusion from analytics. The evidence appendix records the actual final test count and elapsed time. A test checklist without execution is not presented as a passed test.

Browser review covers desktop and narrow layouts, sign-in, cohort selection, generation, timetable review and publication confirmation. Screenshots in the submission package show the implemented UI. The complete ER and dictionary are extracted from the application schema. Oracle companion execution is separately verified on Oracle 21c; its results are kept distinct from the local MySQL/MongoDB test suite.

8. Demonstration plan and viva preparation

Eight-minute demonstration

1. Introduce the scheduling problem and three resource constraints: faculty, division and room.

2. Show the ER overview and explain the faculty_courses junction plus composite eligibility foreign key.

3. Explain one normalization example: RoomID determines capacity, so room facts belong in rooms rather than every timetable entry.

4. Open the academic assignments and generate semester 3 in a fresh installation (or semester 5 if the prepared semester 3 example is already published). Point out the two-hour practical blocks and separate-day course sessions.

5. Publish the run. Show the audit history and explain the procedure-trigger-transaction chain.

6. Generate semester 5 and explain how existing published faculty and room reservations are respected.

7. Run the live savepoint/rollback experiment; show 1, 1, 0 and the installed trigger inventory.

8. Show the EXPLAIN comparison and automated test report. End with limitations and feasible future work.

Viva questions and concise answers

Question	Answer
Why use a relational database?	Shared academic facts have explicit relationships, keys and consistency rules; transactions keep related changes atomic.
Why a junction table?	Faculty and courses are many-to-many. The pair key prevents duplicate eligibility records.
Why a composite foreign key in offerings?	Two separate foreign keys prove that a faculty and course exist; the composite reference proves that the faculty is eligible for that course.
What is 3NF?	For each nontrivial dependency X -> A, X is a superkey or A is prime. The documented business relations avoid non-key transitive dependencies.
Why keep default and offering weekly counts?	A subject default and a division-specific requirement are different facts. Scheduling uses the latter.
Procedure versus trigger?	A procedure is called explicitly, for example publication. A trigger runs automatically on its table event, for example audit insertion.
Why validate both in Python and SQL?	Python avoids bad placements efficiently; SQL protects persisted integrity and rechecks at the publication boundary.
What happens after the fifth insert fails?	The transaction context rolls back earlier inserts, including the new run.
What does FOR UPDATE do?	It locks selected rows until transaction completion so competing supported writers serialize.
Why validate again at publish?	Another timetable may have been published after a draft was generated or displayed.
Why not COMMIT inside the procedure?	The caller must control the entire atomic unit, including related changes and audit work.
Why is the composite index useful?	Equality predicates on its leading columns narrow the search to matching entries instead of scanning the table.
Does an index always improve a query?	No. Selectivity, table size, statistics, write cost and query shape matter. EXPLAIN must be inspected.
Is the schedule optimal?	It is a feasible solution under the declared constraints; no global optimality claim is made.
Why keep session_no?	It distinguishes one multi-period session from another and allows completeness/contiguity checks.
How is room utilization computed?	Published occupied hours divided by teaching periods times six days, for the selected academic year.
What is a savepoint?	A named point inside a transaction that permits partial rollback without ending the transaction.
Is MySQL PL/SQL?	No. This application uses MySQL stored SQL. The separate Oracle lab uses PL/SQL syntax and must be run on Oracle.

9. Limitations, future work and references

This local project supports fixed one-hour periods, one teacher per offering, six teaching days, up to six sessions per offering per week and at most 200 sessions per generated run. It omits calendar-date exceptions, multi-campus travel, student electives, team teaching, multi-role portals and optimization objectives such as minimizing gaps. Saved academic editions intentionally constrain catalog edits to preserve history. A frozen edition is replaced using new academic records, not silently rewritten.

Future work could add date-bounded terms, faculty-specific preference scoring, CP-SAT optimization, dedicated approval roles, schema migrations and scoped reservation locks. These are extensions, not features claimed as complete.

References: MySQL 8.4 Reference Manual (https://dev.mysql.com/doc/refman/8.4/en/); Connector/Python transaction documentation (https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqlconnection-start-transaction.html); Oracle PL/SQL error handling (https://docs.oracle.com/en/database/oracle/oracle-database/21/lnpls/plsql-error-handling.html); Graphviz (https://graphviz.org/). The supplied source, metadata dictionary, test output and benchmark JSON are the primary evidence for project-specific claims.

CampusWeave: complete syllabus companion

Vishwakarma Institute of Technology | Computer Engineering | AY 2026-27

This workbook connects every supplied syllabus topic to the timetable project. It distinguishes a working feature, an executed lab, a local simulation and an explanatory design. A simulation is not described as a deployed cluster. The Oracle scripts were separately executed on Oracle 21c in an approved CC0 db<>fiddle session; this does not imply that the Flask application uses Oracle.

Unit 1: database foundations and modeling

Need, evolution, concepts and design process

The starting file-based solution duplicates faculty names, room capacities and assignments across spreadsheets. A DBMS centralizes integrity, concurrent access, query processing and recovery. The evolution from hierarchical/network systems to relational systems, object-relational features and distributed NoSQL reflects changing workloads rather than a single universally best model.

CampusWeave uses SQL for authoritative scheduling facts and MongoDB for flexible feedback documents. Schema is the structure; an instance is its current data. Logical independence separates relational structure from application views; physical independence permits indexes without changing the query's meaning. External views include the faculty schedule; the conceptual layer contains entities/relationships; the internal layer contains InnoDB indexes, pages and logs.

The design process is requirements -> conceptual ER -> logical relations -> normalization -> physical indexes -> implementation -> verification. PROJECT_REPORT.md follows this sequence. ER_complete.svg covers the operational schema; ER_warehouse.svg covers the dimensional mart.

ER, extended ER and relational model

Entities have keys and attributes; relationships have cardinality and participation. Faculty-course eligibility is M:N and becomes a junction table. Faculty's user-account participation is optional. A course offering participates in one division, one course and one eligible teacher pair.

An extended-ER interpretation specializes User into Admin, HOD, Coordinator, Faculty and Student by the role discriminator. The current enum stores one role per account, corresponding to disjoint specialization; all users require a role, making the discriminator total. Only the authenticated coordinator-style scheduling workflow is implemented; separate subtype portals are not claimed. If overlapping roles were needed, user_roles(user_id,role_id) would replace the single discriminator. Aggregation can treat a course offering (division-course-teacher association) as the object scheduled into a period.

The relational model represents tuples, domains, keys and referential integrity. SQL tables are bags unless uniqueness or DISTINCT removes duplicates. NULL represents missing/unknown values and requires IS NULL rather than equality comparisons.

Codd's rules: a project interpretation

Rule	Meaning and connection
0	Relational management: scheduling facts are managed through relational capabilities.
1	Information rule: facts are represented as values in relations.
2	Guaranteed access: table, primary key and column identify a value.
3	Systematic NULL handling: optional user linkage uses NULL explicitly.
4	Relational catalog: information_schema supplies the dictionary and diagrams.
5	Comprehensive language: SQL provides definitions, queries, integrity, privileges and transactions.
6	View updating: a simple checked view is updated in the SQL lab; arbitrary joined/aggregate views are not automatically updatable.
7	High-level operations: set-based UPDATE/DELETE and aggregation operate on multiple rows.
8	Physical independence: adding an index does not change query results.
9	Logical independence: views can hide some schema changes; not every change is transparent.
10	Integrity independence: keys/checks/triggers live in the database.
11	Distribution independence: a future distributed layer should preserve logical query behavior; the local system does not implement this rule across machines.
12	Nonsubversion: application routes cannot bypass SQL guards; unrestricted administrator access is outside that guarantee.

This mapping explains the rules; it is not a claim that MySQL fully satisfies every Codd rule.

Unit 2: dependencies, normalization and synthesis

The project report derives 1NF, 2NF and 3NF and lists actual candidate keys. Atomic fields eliminate repeating groups. Separating offering/run facts removes partial dependencies. Separating faculty, course, room and department descriptions removes transitive dependencies.

Armstrong inference, closure and minimal cover

Reflexivity: if Y is a subset of X, then X -> Y. Augmentation: X -> Y implies XZ -> YZ. Transitivity: X -> Y and Y -> Z imply X -> Z. Union, decomposition and pseudotransitivity follow from these axioms.

The executed example has F = {A -> B, B -> C, A -> C, AB -> D}. Closure A+ is {A,B,C,D}, so A is a candidate key. In AB -> D, B is extraneous because A already determines B. A -> C is redundant through B. The minimal cover is {A -> B, B -> C, A -> D}. syllabus_lab.py computes these results rather than storing them as claimed outputs.

The 3NF synthesis groups dependencies by determinant, creates AB D and BC relations, removes subsumed relations and adds a candidate-key relation when none is present. For this example the output is R1(A,B,D), R2(B,C).

It is dependency-preserving and lossless under the synthesis theorem because a key relation is retained. General decomposition is not automatically lossless merely because its tables have primary keys.

Lossless join and dependency preservation

For a binary decomposition R -> R1,R2 under FDs, losslessness follows when the common attributes determine R1 or R2. Dependency preservation means the union of dependencies projected onto components implies the original FDs without joining relations. These are separate properties. The report's entity decompositions retain the determining key as the shared attribute.

BCNF and its tradeoff

BCNF requires every determinant of a nontrivial FD to be a superkey. Consider R(Student,Course,Instructor), with (Student,Course) -> Instructor and Instructor -> Course. Candidate keys are (Student,Course) and (Student,Instructor). This can satisfy 3NF because Course is prime, but violates BCNF because Instructor is not a superkey. Decompose into InstructorCourse(Instructor,Course) and StudentInstructor(Student,Instructor). The decomposition is lossless because the intersection Instructor determines InstructorCourse, but the original (Student,Course) -> Instructor is not enforceable in either component alone. This is an educational counterexample, not an unannounced replacement of CampusWeave's schema.

MVD and fourth normal form

Suppose a teacher's preferred days and preferred labs are independent: Faculty ->> Day and Faculty ->> Lab. Storing every (Faculty,Day,Lab) combination creates a cross product. Decompose into FacultyDay and FacultyLab to reach 4NF when the nontrivial MVD determinant is not a superkey. The executed lab reconstructs four combinations from two day rows and two lab rows and verifies equality. Independence is an explicit assumption; real course/lab compatibility must not be decomposed as if it were independent.

Unit 3: query languages and procedural constructs

Relational algebra to SQL

Algebra expression (plain-text notation)	Equivalent SQL idea
sigma capacity >= 60 (rooms)	SELECT * FROM rooms WHERE capacity>=60
pi room_code (rooms)	SELECT DISTINCT room_code FROM rooms
faculty JOIN department_id departments	Equijoin with ON or USING(department_id)
eligible_for_DBMS UNION eligible_for_DSA	Two compatible SELECT projections joined with UNION
all_faculty MINUS faculty_with_offerings	EXCEPT, or a correlated NOT EXISTS
rooms PRODUCT divisions followed by capacity selection	Non-equijoin ON rooms.capacity>=divisions.student_count
gamma subject_type, COUNT(*) (courses)	GROUP BY subject_type with COUNT(*)

Projection in classical algebra removes duplicates; plain SELECT does not. SQL's NULL/three-valued logic also differs from classical two-valued relational algebra. The executable SQL showcase uses DISTINCT and explicit NULL tests where needed.

SQL coverage and execution evidence

sql/07_syllabus_queries.sql executes 50 statements in an isolated database. It covers CREATE/ALTER, INSERT/UPDATE/DELETE, logical/conditional operators, LIKE/NOT LIKE, IN/NOT IN, BETWEEN, IS NULL/IS NOT NULL, ORDER BY, GROUP BY, aggregates and HAVING. It includes UNION/UNION ALL/INTERSECT/EXCEPT; string/date/time/numerical functions; equi/non-equi/self/outer joins; scalar/multirow/correlated subqueries; ANY/ALL/EXISTS; and DML with a subquery.

The simple lab_room_view updates its base table and WITH CHECK OPTION rejects a row that would leave the view. The multi-table faculty_schedule_view provides a reporting abstraction. Aggregation, DISTINCT, grouping and other constructs impose view-update restrictions; not every join view is universally read-only in every DBMS, so use the metadata and engine's rules rather than a blanket claim.

DCL, roles, assertions, embedded and dynamic SQL

The SQL experiment creates a temporary non-login role, grants SELECT on its isolated schema, records SHOW GRANTS, revokes the privilege and removes the role. The actual app account is database-scoped; administrative passwords are kept in ignored runtime files. Role metadata in users is not the same thing as a database role.

TCL is exercised through START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT and ROLLBACK TO SAVEPOINT. SQL-standard CREATE ASSERTION is not supported by MySQL; the cross-row timetable assertion is implemented using procedures, triggers and locking. A trigger without a concurrency protocol alone would not prove race-free enforcement.

Python's connector calls are host-language database connectivity with bound parameters; they are not a precompiled EXEC SQL embedded-SQL language. Traditional embedded SQL uses host variables, a precompiler and SQLCA/status handling. The project provides the equivalent host-program query workflow through DB-API. The dynamic-SQL lab uses PREPARE, EXECUTE USING and DEALLOCATE; the UI's identifiers come from a fixed allowlist and values remain parameters.

PL/SQL procedure, function, cursors and trigger types

The primary application executes MySQL procedures, triggers and fn_run_hours. Oracle syntax is supplied separately in sql/oracle/plsql_lab.sql and advanced_plsql.sql. The companion includes a stored publication procedure, utilization function, anonymous blocks, exception handling and DBMS_OUTPUT.

Cursor coverage includes implicit SQL cursor attributes; explicit OPEN/FETCH/CLOSE; parameterized cursors; cursor FOR loops; weak SYS_REFCURSOR and strong REF CURSOR types; and bound dynamic cursor SQL. Oracle also has FOR UPDATE cursors and WHERE CURRENT OF for row updates; an example is supplied below. These scripts were executed on Oracle 21c; oracle_execution.md records cursor trace rows, function results, expected exception handling and empty USER_ERRORS results.

Trigger coverage includes BEFORE row validation; AFTER INSERT/UPDATE/DELETE row events; BEFORE statement events; a compound trigger accumulating an affected-row count; AFTER statement processing; and INSTEAD OF UPDATE on a join view. Optional schema DDL auditing is shown but not installed. Database LOGON/LOGOFF/STARTUP/SHUTDOWN triggers require administrator scope and are intentionally not installed on a shared database.

Unit 4: query processing, transactions, concurrency and recovery

Query processing parses and validates SQL, rewrites equivalent expressions, estimates costs from statistics, selects access paths/join orders and executes a physical plan. Predicate pushdown and projection pruning reduce work; indexed nested loops, hash joins or scans suit different cardinalities. Selecting every column can prevent a covering index from satisfying a query alone.

The 60,000-row controlled experiment compares the same result with and without a composite index. EXPLAIN changes from ALL/60,000 estimated rows to ref/one estimated row. evidence.json includes the complete plans and seven-sample medians. This is a reproducible microbenchmark, not a production capacity forecast.

Transaction state diagram

Active -> Partially committed -> Committed is the successful path. A failure from Active or Partially committed leads to Failed -> Aborted, followed by restart or termination. A Python exception before commit invokes rollback. A database crash during commit requires recovery to decide the durable outcome; a client timeout does not by itself prove rollback.

ACID is demonstrated in the report: all-or-nothing persistence, constraint consistency, serialized writes and committed storage. Schedules interleave operations from multiple transactions. Conflict equivalence preserves the order of conflicting same-item operations from different transactions when at least one is a write. A precedence graph is conflict-serializable exactly when it is acyclic.

The executable graph lab gives a serializable T1 -> T2 example and a cyclic T1 <-> T2 example. View equivalence preserves initial reads, the precise write each read observes and final writers. W1(X), W2(X), W1(X) is view-serializable to T2,T1 but not conflict-serializable because of blind writes. The checker enumerates at most six transactions; it is a teaching tool, not a large schedule verifier.

Concurrency protocols

Two-phase locking has a growing lock-acquisition phase and a shrinking release phase. Strict 2PL holds exclusive locks until commit/abort and simplifies recovery. CampusWeave's InnoDB row lock serializes supported writers until transaction completion. Timestamp ordering rejects operations that violate timestamp order; optimistic concurrency validates at commit; MVCC provides versioned reads. These alternatives are explained, not falsely claimed as custom implementations in this app.

Deadlocks require a cycle of waiting transactions and can be detected or prevented through consistent lock ordering. The app takes its global scheduling lock first. Concurrent publication tests prove that two drafts for one cohort cannot both win. Snapshot reads alone do not eliminate write skew; publication must revalidate under the write protocol.

Recovery techniques

Write-ahead logging requires the relevant log information to be durable before a changed page is flushed. Undo restores uncommitted updates; redo reapplies committed changes that may not have reached data pages. Checkpoints bound recovery scanning. Shadow paging is an alternative approach with different fragmentation/concurrency tradeoffs. Backups plus log replay address failures beyond a single transaction.

The executed simplified recovery model undoes uncommitted T2's faculty-limit change and redoes committed T1's capacity change. Its recovered values are 16 and 65. This is not a claim of implementing ARIES or InnoDB crash recovery, nor a power-loss test.

Unit 5: NoSQL, big data, Hadoop and warehousing

NoSQL types, BASE and CAP

Key-value stores suit direct key lookup; document stores suit nested/variable records; wide-column stores suit partitioned sparse data; graph stores emphasize relationship traversal. Faculty feedback uses MongoDB documents with a tag array, rating, note and timestamps. The relational schedule remains the source of truth; no distributed transaction spans MySQL and MongoDB.

BASE means Basically Available, Soft state, Eventually consistent and describes a design tendency, not a mandatory property of every NoSQL product. MongoDB also supports stronger consistency and transactions in suitable topologies. CAP concerns the choice between linearizable consistency and availability during a network partition, not

an unrestricted 'choose any two' rule. The local CAP simulation explicitly returns stale data in its AP branch and refuses a read in its CP branch, then reconciles replicas.

Actual MongoDB work

The authenticated local MongoDB 8.0 database stores faculty_feedback. The Flask feedback screen implements create/read/update/delete through PyMongo and a live aggregation by subject. The isolated lab executes 16 operations: insertMany, find/count, equality, comparison, projection, array membership, sorting/limit, updateOne, updateMany, deleteOne, distinct, group/average/sum, compound indexing, hinted explain and regex filtering, plus actual mapReduce.

The legacy MapReduce result is compared with aggregation totals. MongoDB has deprecated MapReduce since version 5.0; it remains in this teaching lab only because the syllabus requests it. Production-oriented code uses the aggregation pipeline. See mongo_evidence.json for execution results and the official command reference at https://www.mongodb.com/docs/manual/reference/command/mapreduce/.

Big data, HDFS and MapReduce

Big data discussions consider volume, velocity, variety, veracity and value. A timetable demo is not a big-data workload. HDFS separates NameNode metadata from DataNode block storage and replicates blocks for fault tolerance. The provided model places three replicas per block and calculates remaining replicas after a node failure. A real deployment also needs heartbeats, re-replication, rack awareness and operational management.

MapReduce maps input to key-value pairs, partitions and shuffles by key, then reduces grouped values. A combiner may reduce network traffic when its operation is valid. The local parallel experiment maps department fragments to hour totals, combines results and verifies equality with a serial calculation. Hadoop Streaming mapper/reducer scripts are supplied for use with a real Hadoop installation; they are also tested locally. No Hadoop cluster has been installed or benchmarked on this PC.

Warehouse architecture and OLAP

The pipeline is operational sources -> extraction/staging -> transformation -> dimensional warehouse -> OLAP/reporting. The implemented MySQL teaching mart has dw_teaching_fact at one occupied published period per source entry, with subject, faculty, cohort and period dimensions. Idempotent loading uses the source entry key; repeated loads do not duplicate facts.

The star stores descriptive dimensions directly. A snowflake would normalize dimension hierarchies such as department; a fact constellation would share dimensions across teaching and room-maintenance facts. The current mart is a star, not all three schemas simultaneously. Dimensions are first-load snapshots; a production mart would define slowly changing dimensions and effective dates.

The warehouse screen performs ROLLUP across academic year, semester and subject type. Slice chooses a single dimension value; dice chooses multiple restrictions; drill-down shows finer detail; roll-up aggregates; pivot rotates categorical values into columns. Historical editions remain in the mart after archival, so its totals describe edition history and must not be confused with current room occupancy.

Unit 6: parallel and distributed databases

Shared-memory architectures coordinate through common memory; shared-disk systems share storage with separate memory; shared-nothing systems partition both compute and storage. Local threads are not evidence of a shared-nothing cluster. I/O parallelism spreads reads across disks/partitions. Interquery parallelism runs independent queries simultaneously. Intraquery parallelism divides one query; intraoperation parallelism parallelizes one operator, while interoperation parallelism pipelines or overlaps different operators.

The local thread-pool aggregation illustrates intraoperation parallelism over two fragments. Concurrent publication tests illustrate independent client operations contending for one transactional resource. No speedup is claimed for

Python threads; the purpose is decomposition and correctness.

Homogeneous distributed databases use the same engine; heterogeneous/federated systems integrate differing engines and schemas. MySQL plus MongoDB here is polyglot persistence on one PC, not a transparent federated distributed DBMS. Horizontal fragmentation partitions rows by a predicate; vertical fragmentation splits columns while retaining the reconstruction key; hybrid fragmentation combines them. Replication makes copies; partitioning splits data. The executable lab verifies horizontal union and vertical key-join reconstruction.

Distributed query processing localizes a global query to fragments, pushes filters/projections down, chooses join locations and may use semijoins to reduce transfer. Network cost and skew can dominate local CPU cost. A two-phase commit coordinator asks participants to prepare, then records and distributes commit/abort; prepared participants may block if the coordinator fails. CampusWeave deliberately does not pretend that independent MySQL and MongoDB writes form an atomic distributed transaction.

Laboratory experiment checklist

Lab	Supplied implementation / evidence	Verification status
1 DDL and DML	01_schema.sql, 07_syllabus_queries.sql	Executed on MySQL; constraints and rollback tested.
2 SELECT operators/functions/sets	07_syllabus_queries.sql	Executed; output samples in sql_evidence.json.
3 Joins/subqueries/DML subqueries/strategies	Query showcase, SQL lab, EXPLAIN benchmark	Executed.
4 Views/base updates/restrictions	lab_room_view, active_room_catalog, reporting view	Base update and CHECK OPTION rejection executed.
5 PL/SQL cursors	oracle/advanced_plsql.sql	Executed on Oracle 21c; six cursor variants produce trace rows.
6 Procedures/functions	MySQL procedures and fn_run_hours; Oracle companion	MySQL and Oracle 21c companion executed.
7 Trigger types	MySQL integrity/audit triggers; Oracle row/statement/compound/INSTEAD OF scripts	MySQL and Oracle 21c companion executed.
8 MongoDB queries/aggregation/indexing/ MapReduce	mongo_experiments.py; mongo_evidence.json	16 operations executed, including actual legacy MapReduce.
9 Relational connectivity	Flask + mysql-connector-python	Working application and integration tests.
10 Non-relational connectivity	Flask + PyMongo feedback module	Actual CRUD and aggregation tested.

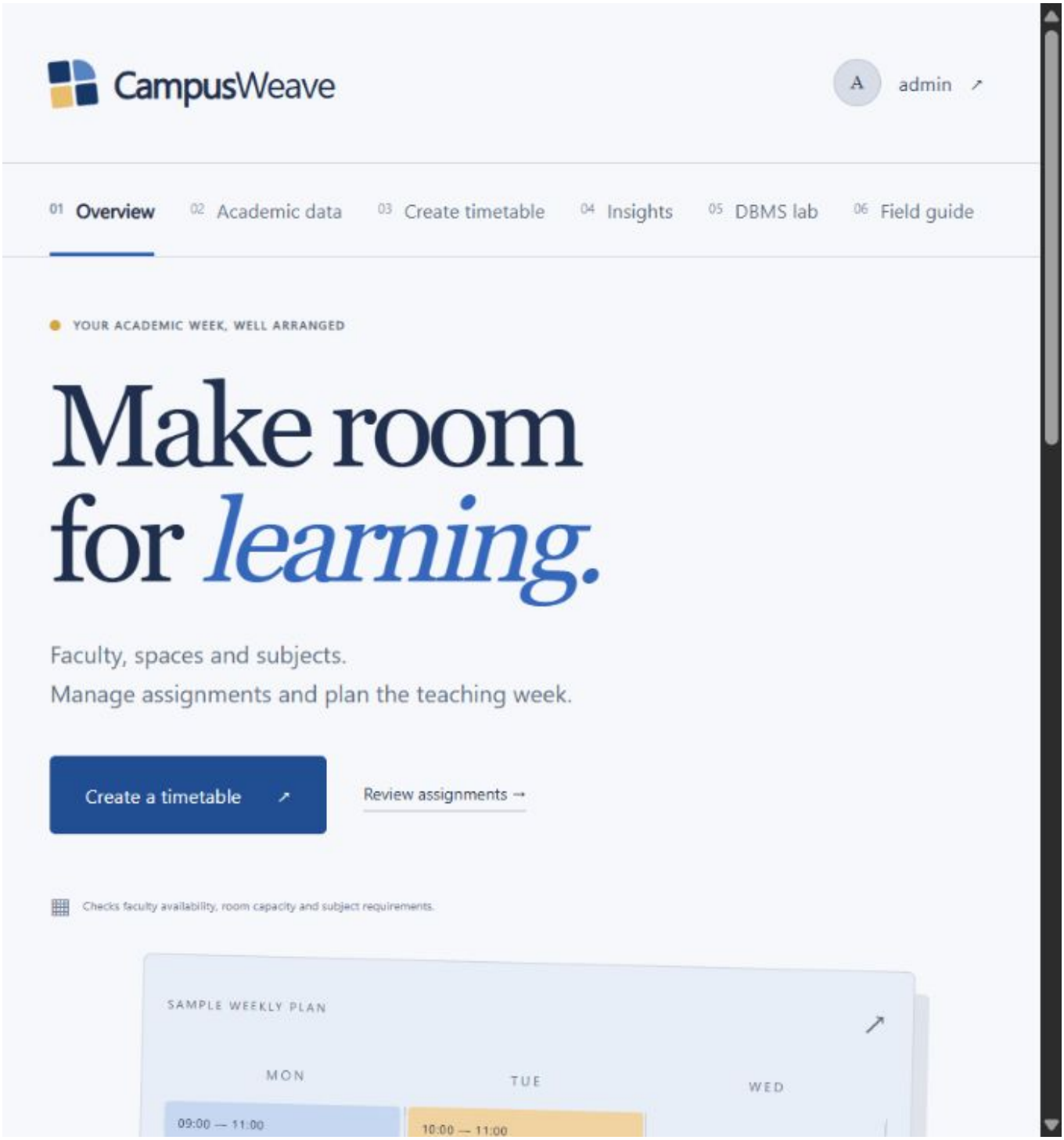
Two-tier and three-tier architecture

The normal browser -> Flask application -> MySQL/MongoDB flow is a logical three-tier architecture even though all tiers run on one PC. A two-tier example is scripts/two_tier_report.py, a Python client that directly connects to MySQL and prints faculty workload. Physical deployment count and logical tier count are different concepts.

Evidence and honesty boundaries

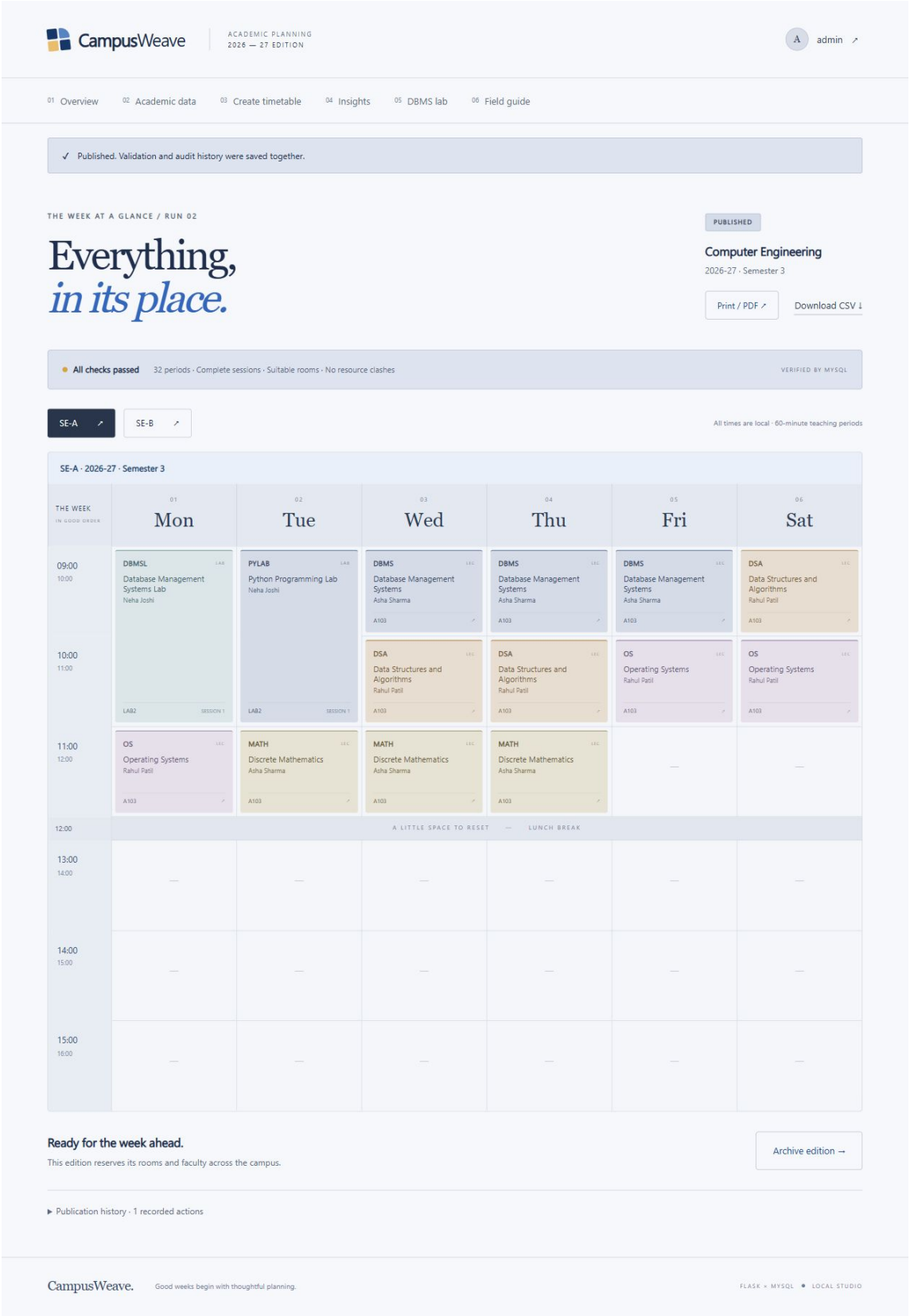
The project demonstrates the complete supplied topic list through implemented features, executed labs, executed Oracle source and explicitly labeled teaching models. A real Hadoop/distributed cluster is not represented as deployed. The code and workbook give the examiner concrete material for each topic without destabilizing the simple faculty planner.

The faculty workspace



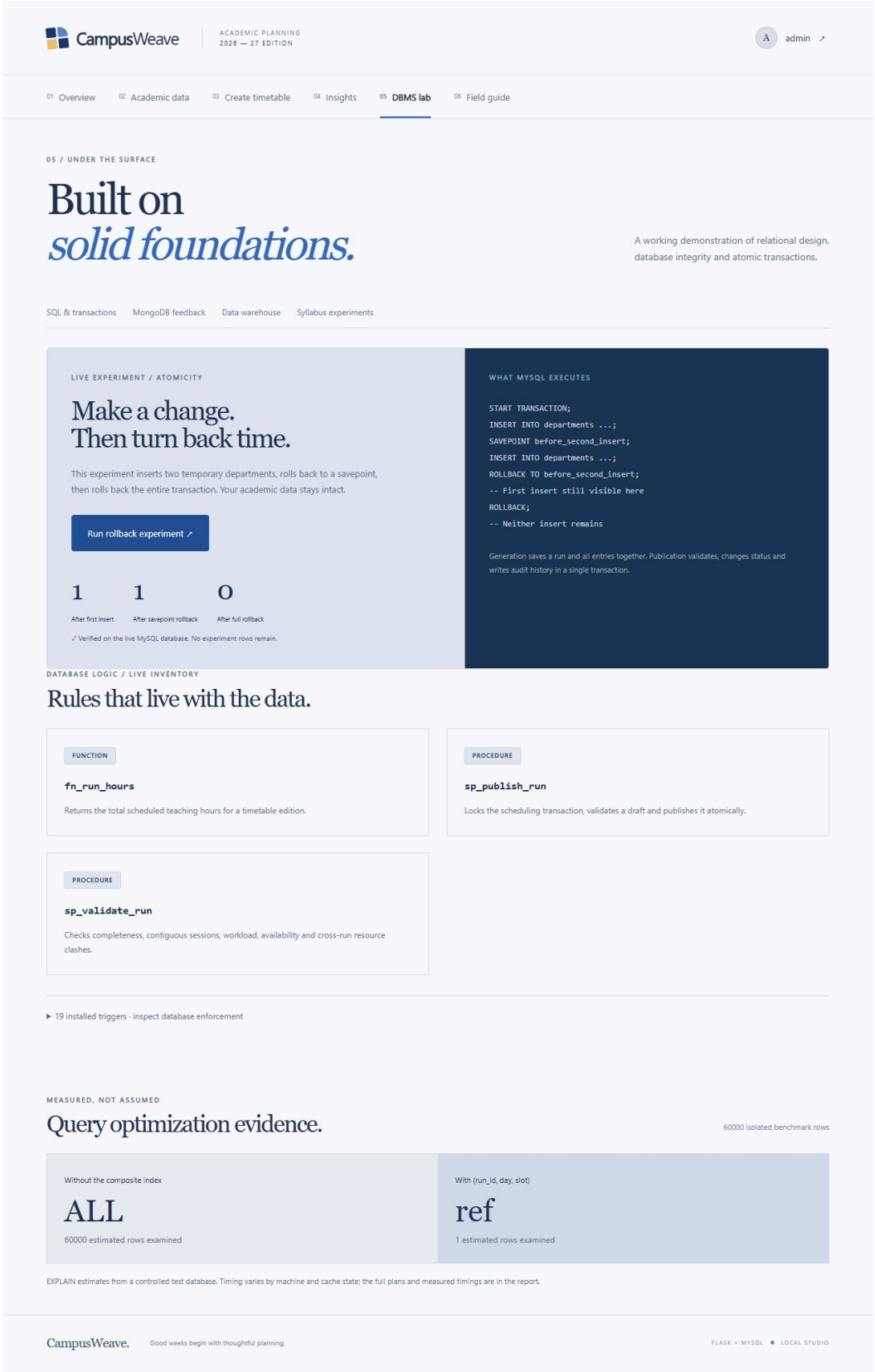
Source: CampusWeave implementation. Full-resolution SVG and metadata dictionary are included in the repository.

A published academic week



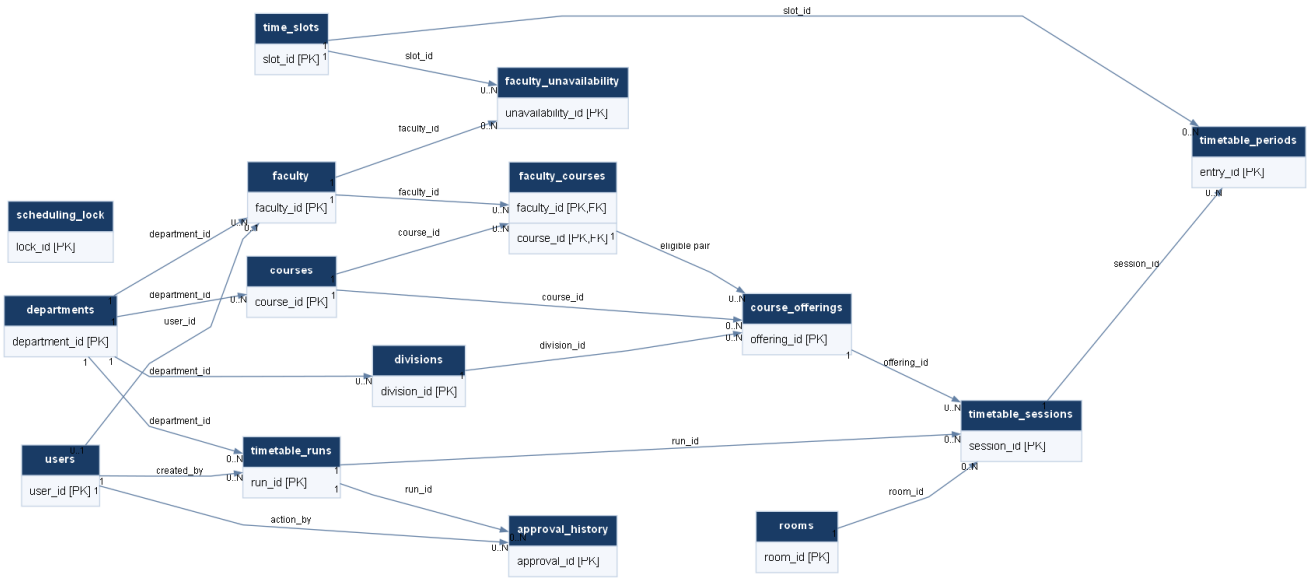
Source: CampusWeave implementation. Full-resolution SVG and metadata dictionary are included in the repository.

Live transaction and trigger evidence



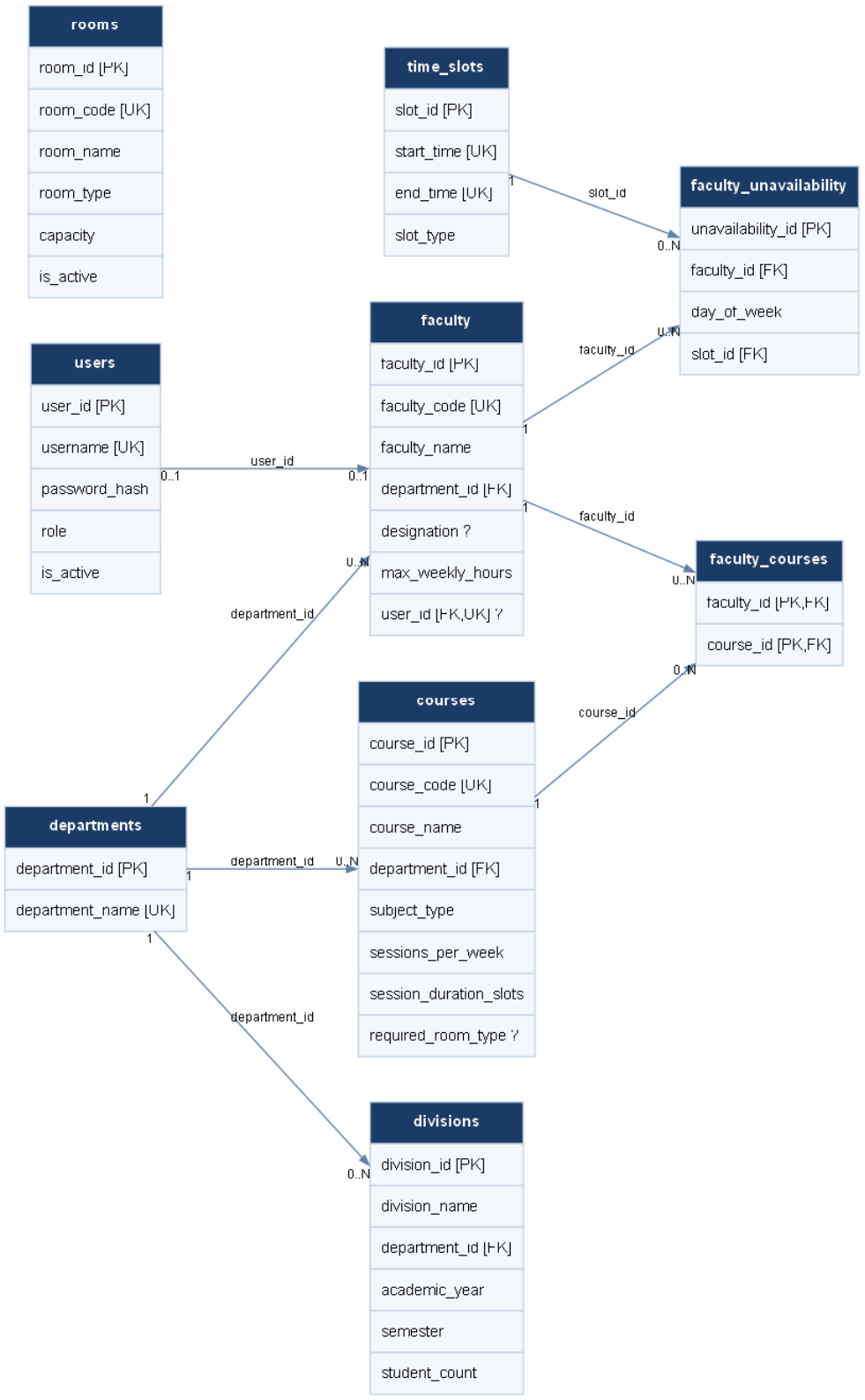
Source: CampusWeave implementation. Full-resolution SVG and metadata dictionary are included in the repository.

Operational ER overview



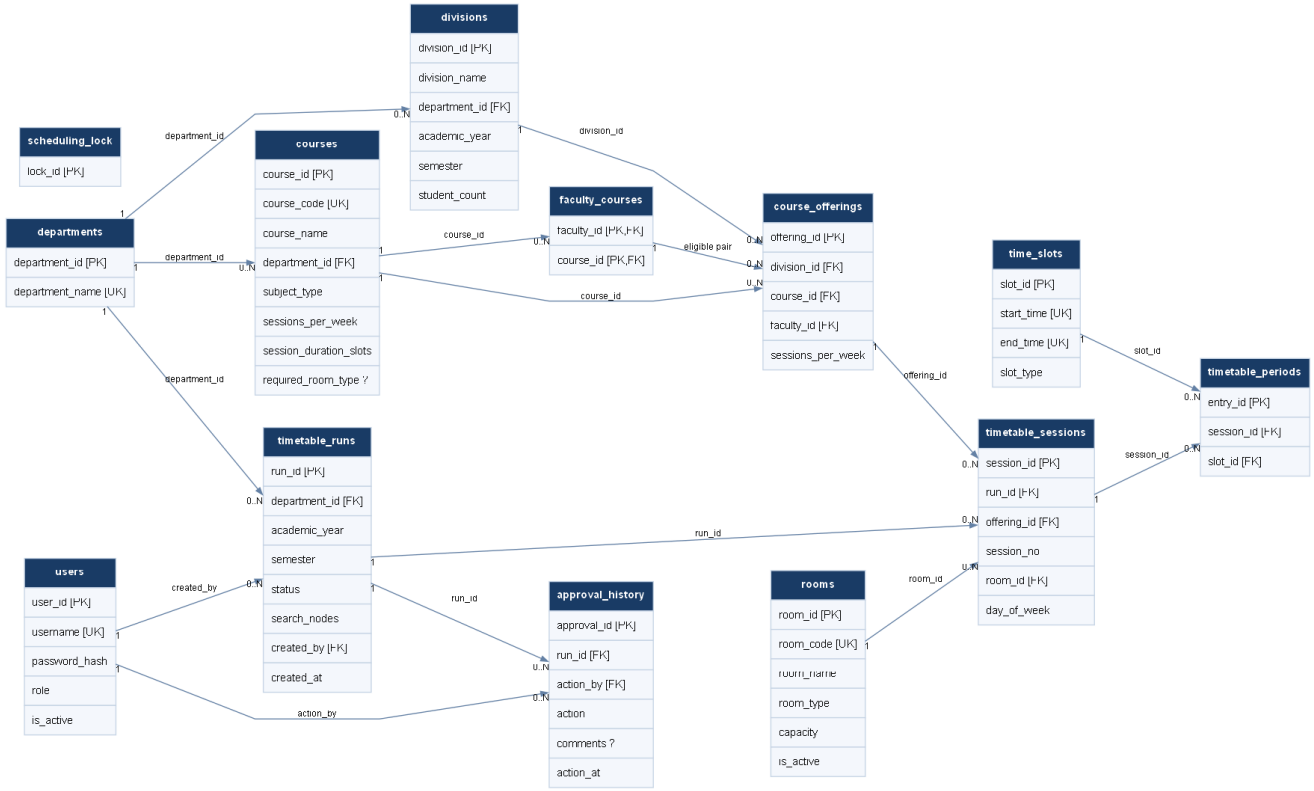
Source: CampusWeave implementation. Full-resolution SVG and metadata dictionary are included in the repository.

Academic entities and relationships



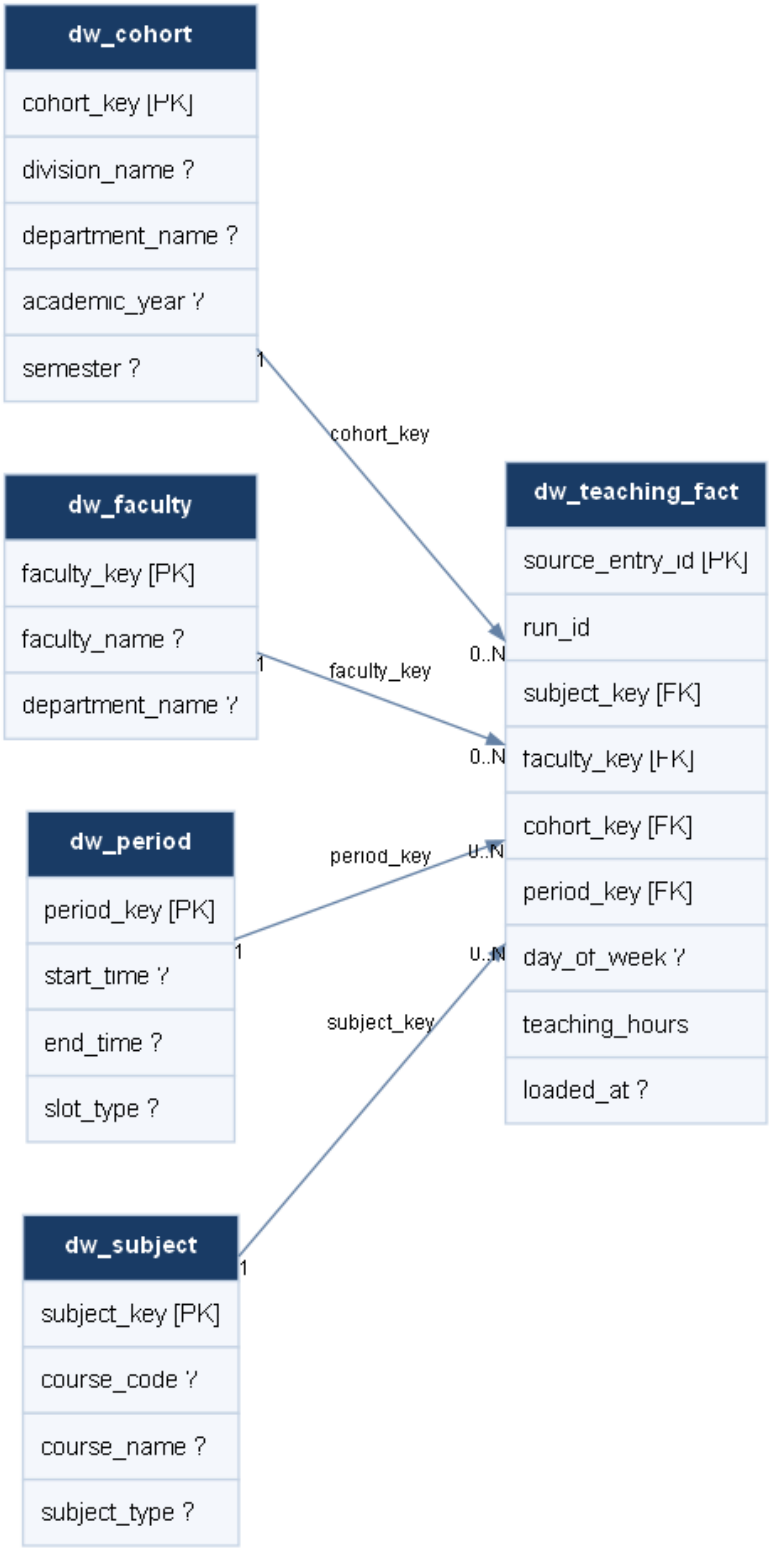
Source: CampusWeave implementation. Full-resolution SVG and metadata dictionary are included in the repository.

Scheduling entities and relationships



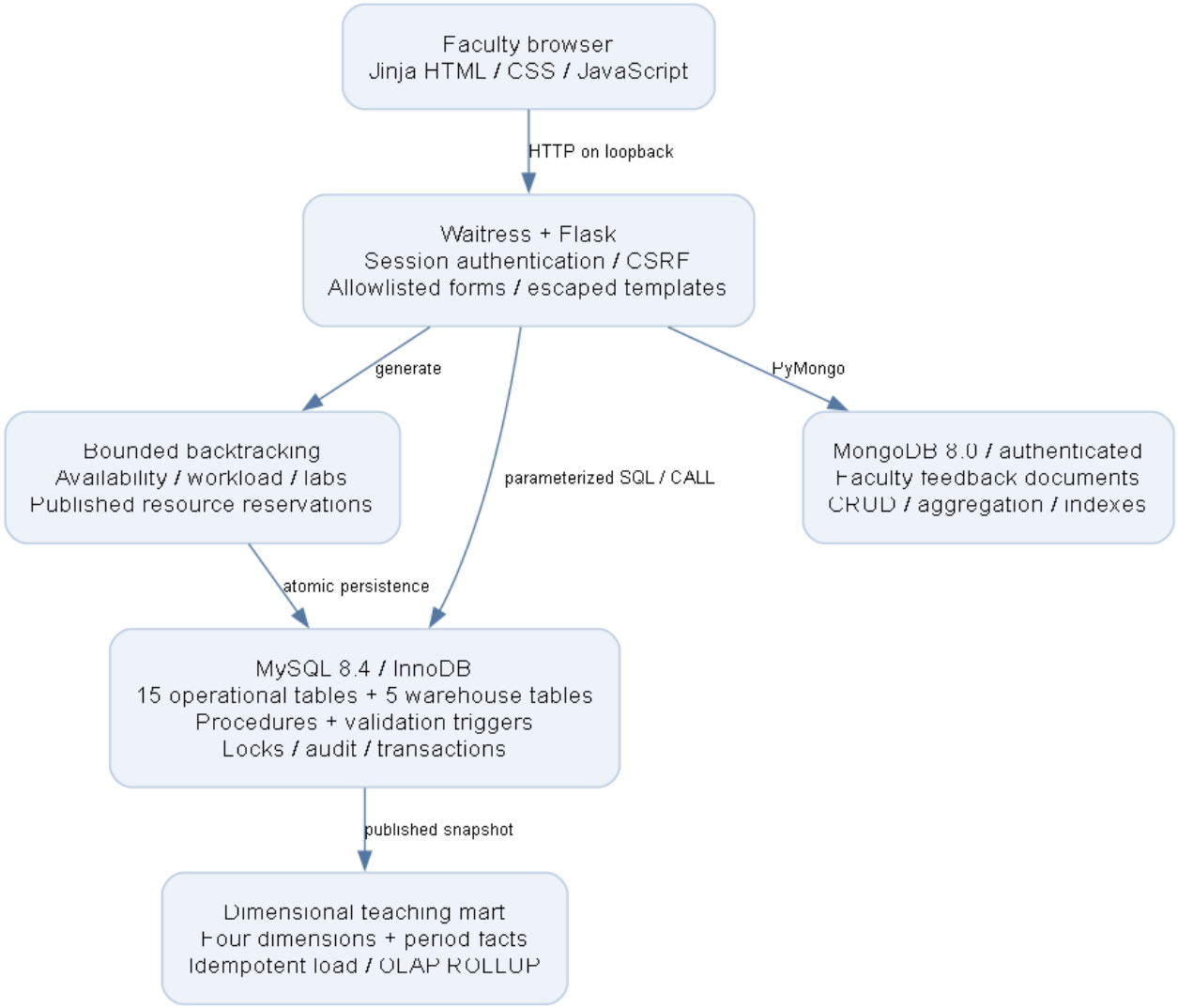
Source: CampusWeave implementation. Full-resolution SVG and metadata dictionary are included in the repository.

The warehouse star schema



Source: CampusWeave implementation. Full-resolution SVG and metadata dictionary are included in the repository.

Application architecture



Source: CampusWeave implementation. Full-resolution SVG and metadata dictionary are included in the repository.

Data dictionary

PK: primary key; UNI: unique key; MUL: indexed/referencing column. See schema_dictionary.json and SQL for complete composite keys, foreign keys and CHECK constraints. The timetable_entries relation is a joined view over sessions and periods.

approval_history

Column	Type	Null?	Key
approval_id	int	NO	PRI
run_id	int	NO	MUL
action_by	int	NO	MUL
action	enum('PUBLISHED','ARCHIVED')	NO	-
comments	varchar(500)	YES	-
action_at	timestamp	NO	-

course_offerings

Column	Type	Null?	Key
offering_id	int	NO	PRI
division_id	int	NO	MUL
course_id	int	NO	MUL
faculty_id	int	NO	MUL
sessions_per_week	int	NO	-

courses

Column	Type	Null?	Key
course_id	int	NO	PRI
course_code	varchar(20)	NO	UNI
course_name	varchar(100)	NO	-
department_id	int	NO	MUL
subject_type	enum('THEORY','PRACTICAL','TUTORIAL')	NO	-
sessions_per_week	int	NO	-
session_duration_slots	int	NO	-
required_room_type	enum('CLASSROOM','COMPUTER_LAB','ELECTRONICS_LAB','OTHER_LAB')	YES	-

departments

Column	Type	Null?	Key
department_id	int	NO	PRI
department_name	varchar(100)	NO	UNI

divisions

Column	Type	Null?	Key
division_id	int	NO	PRI
division_name	varchar(30)	NO	MUL
department_id	int	NO	MUL
academic_year	varchar(20)	NO	-
semester	int	NO	-
student_count	int	NO	-

dw_cohort

Column	Type	Null?	Key
cohort_key	int	NO	PRI
division_name	varchar(30)	YES	-
department_name	varchar(100)	YES	-
academic_year	varchar(20)	YES	-
semester	int	YES	-

dw_faculty

Column	Type	Null?	Key
faculty_key	int	NO	PRI
faculty_name	varchar(100)	YES	-
department_name	varchar(100)	YES	-

dw_period

Column	Type	Null?	Key
period_key	int	NO	PRI
start_time	time	YES	-
end_time	time	YES	-
slot_type	varchar(20)	YES	-

dw_subject

Column	Type	Null?	Key
subject_key	int	NO	PRI
course_code	varchar(20)	YES	-
course_name	varchar(100)	YES	-
subject_type	varchar(20)	YES	-

dw_teaching_fact

Column	Type	Null?	Key
source_entry_id	int	NO	PRI
run_id	int	NO	-
subject_key	int	NO	MUL
faculty_key	int	NO	MUL
cohort_key	int	NO	MUL
period_key	int	NO	MUL
day_of_week	varchar(12)	YES	-
teaching_hours	decimal(4,1)	NO	-
loaded_at	timestamp	YES	-

faculty

Column	Type	Null?	Key
faculty_id	int	NO	PRI
faculty_code	varchar(20)	NO	UNI
faculty_name	varchar(100)	NO	-
department_id	int	NO	MUL
designation	varchar(50)	YES	-
max_weekly_hours	decimal(4,1)	NO	-
user_id	int	YES	UNI

faculty_courses

Column	Type	Null?	Key
faculty_id	int	NO	PRI
course_id	int	NO	PRI

faculty_unavailability

Column	Type	Null?	Key
unavailability_id	int	NO	PRI
faculty_id	int	NO	MUL
day_of_week	enum('MONDAY','TUESDAY','WEDNESDAY','THURSDAY','FRIDAY','SATURDAY')	NO	-
slot_id	int	NO	MUL

rooms

Column	Type	Null?	Key
room_id	int	NO	PRI
room_code	varchar(20)	NO	UNI
room_name	varchar(100)	NO	-
room_type	enum('CLASSROOM','COMPUTER_LAB','ELECTRONICS_LAB','OTHER_LAB')	NO	-
capacity	int	NO	-
is_active	tinyint(1)	NO	-

scheduling_lock

Column	Type	Null?	Key
lock_id	int	NO	PRI

time_slots

Column	Type	Null?	Key
slot_id	int	NO	PRI
start_time	time	NO	UNI
end_time	time	NO	UNI
slot_type	enum('TEACHING','BREAK')	NO	-

timetable_periods

Column	Type	Null?	Key
entry_id	int	NO	PRI
session_id	int	NO	MUL
slot_id	int	NO	MUL

timetable_runs

Column	Type	Null?	Key
run_id	int	NO	PRI
department_id	int	NO	MUL
academic_year	varchar(20)	NO	MUL
semester	int	NO	-
status	enum('DRAFT','PUBLISHED','ARCHIVED')	NO	-
search_nodes	int	NO	-
created_by	int	NO	MUL
created_at	timestamp	NO	-

timetable_sessions

Column	Type	Null?	Key
session_id	int	NO	PRI
run_id	int	NO	MUL
offering_id	int	NO	MUL
session_no	int	NO	-
room_id	int	NO	MUL
day_of_week	enum('MONDAY','TUESDAY','WEDNESDAY','THURSDAY','FRIDAY','SATURDAY')	NO	-

users

Column	Type	Null?	Key
user_id	int	NO	PRI
username	varchar(50)	NO	UNI
password_hash	varchar(255)	NO	-
role	enum('ADMIN','HOD','COORDINATOR','FACULTY','STUDENT')	NO	-
is_active	tinyint(1)	NO	-